Test Plan 8 — Security (Backend Penetration Testing)

Project	`security`
Specs	`tests/security/backend-pentest.spec.ts`
Target	Organuz Supabase / PostgREST backend (`config.organuzApi`) via the public **anon key**
Client	Browserless `APIRequestContext`; helpers in `tests/security/support/target.ts`
Cases	30
Skips	`SEC-06` and `SEC-08` skip unless a disposable write-probe target is explicitly acknowledged
Skill	none

Scope

Authorized, defensive penetration-testing checks against Organuz's **own** backend, using only the public anon key that ships in the site bundle. The suite is **safe by default**: it reads, targets a guaranteed-missing row, or skips write probes whose failure could mutate data. Against a disposable backend, the two guarded probes can be enabled to verify that INSERT and unqualified DELETE are rejected. There is no fuzzing volume, no DoS, and no service-role key.

The suite asserts the backend's security posture across seven areas: **authentication** (a request must carry a valid key), **authorization / RLS** (the anon key is read-only and cannot reach privileged data), **injection / input handling** (input is treated as data, not executed, and errors don't leak internals), **transport** (HTTPS with a valid certificate, JSON not HTML), **CORS** (no wildcard origin combined with credentials), **key hygiene** (the shipped JWT is the anon role, unexpired, and not `alg:none`), and **account takeover** (`SEC-21…SEC-30`: no forged/tampered token, wrong OTP, or bad credential ever mints a session; no account enumeration; admin user-provisioning is blocked). A **failure here is a real security finding**, not a flake.

All account-takeover probes use a clearly-fabricated identity (`FAKE` in `support/target.ts`), so each can only ever *fail* to authenticate — it never mints a session, never reaches a real account, and never sends a message to a real user.

The group is in the CI matrix (`.github/workflows/parallel-tests.yml`) and runs on every push/PR. Tagged `@security @pentest`.

Preconditions

- Public internet access to the Supabase host, plus the anon key and base URL from `config.json` (`config.organuzApi`). No secrets, no service key.

Cases

`backend-pentest.spec.ts` — "Backend penetration testing (Supabase / PostgREST)" (`@security @pentest`); Allure epic **Security**, feature **Backend penetration testing**.

ID	CASE	ASSERTS	SEVERITY
SEC-01	API rejects a request with no API key	`GET /rest/v1/projects` with no headers → **401**	critical
SEC-02	A garbage API key is rejected	`GET projects` with `apikey: not-a-real-key` → **401**	critical
SEC-03	A malformed bearer JWT is rejected	Valid `apikey` + `Authorization: Bearer garbage.jwt.value` → **401**	critical
SEC-04	The shipped key is the ANON role, never `service_role`	Decoded JWT payload `role === "anon"` (and not `service_role`)	blocker
SEC-05	Baseline public read works	Anon `GET projects?limit=1` → **200**, `application/json`, JSON array	—
SEC-06	Anon INSERT is denied by RLS	Anon `POST projects` → **401/403**, no row created	blocker
SEC-07	Anon UPDATE of a non-existent row is denied or affects zero rows	`PATCH projects?id=eq.<non-existent>` either ≥400 or returns zero updated rows	critical
SEC-08	An unqualified DELETE is refused — no full-table wipe	`DELETE projects` (no filter) → status ≥400 and <500	blocker
SEC-09	A non-exposed table is not dumped	`GET` an unknown table → **401/403/404** (never 200)	—
SEC-10	A privileged auth-admin endpoint is blocked for anon	`GET /auth/v1/admin/users` → **401/403**	critical

ID	CASE	ASSERTS	SEVERITY
SEC-11	Projects rows expose no sensitive fields	No returned row key is in the sensitive-keys list (password, token, secret, api_key, …)	critical
SEC-12	A SQL-injection value in a filter is treated as data, not executed	Injection payload in a filter does not 5xx; a normal read still returns 200 afterward (table survives)	critical
SEC-13	A malformed query returns a JSON error, not an HTML stack trace	4xx, non-HTML content-type, no HTML doc / source path / stack leaked	—
SEC-14	An XSS payload in a query param is not reflected as executable HTML	Payload in `select` → <500 and response is not served as `text/html`	critical
SEC-15	The backend does not 5xx across a batch of malformed requests	Four malformed probes (empty select, non-numeric limit, bad order, bare operator) all return <500	—
SEC-16	The backend is served over HTTPS with a valid certificate	Origin starts with `https://`; a completed request proves the cert chain (Playwright rejects bad certs)	critical
SEC-17	Responses are JSON, never executable HTML	`content-type` matches `application/json`	—
SEC-18	CORS is not unsafe	Cross-origin `OPTIONS` preflight never returns `Allow-Origin: *` together with `Allow-Credentials: true`	critical
SEC-19	The shipped JWT is unexpired and does not use `alg:none`	Header `alg !== "none"`; payload has an `exp` claim that is not already expired	—
SEC-20	Edge functions require the API key	Unauthenticated `GET /functions/v1/...` → ≥400 (never 200)	critical
SEC-21	Password login with forged credentials is rejected	`POST /auth/v1/token?grant_type=password` with the `FAKE` identity → 4xx, never 200, no `access_token` / `refresh_token`	blocker
SEC-22	A forged refresh token cannot be exchanged for a session	`POST token?grant_type=refresh_token` with a bogus token → ≥400, no session issued	blocker

ID	CASE	ASSERTS	SEVERITY
SEC-23	`/auth/v1/user` with a forged bearer JWT is rejected and leaks no profile	Valid `apikey` + forged (bad-signature) user JWT → **401/403**, no `email` / `id` returned	critical
SEC-24	A JWT forged with `alg:none` is rejected	Forged `alg:none` user token on `/auth/v1/user` → **401/403** (signature verification enforced)	blocker
SEC-25	A tampered (elevated-payload) token is refused by the data API	Valid `apikey` + a Bearer JWT re-forged to an authenticated user → **401** on `GET projects`	blocker
SEC-26	OTP verification with a wrong code does not mint a session	`POST /auth/v1/verify` (`type: sms`, fake phone, wrong token) → 4xx, no session	blocker
SEC-27	A failed login is generic — it does not enumerate accounts	Login error body does not disclose account existence (no "user not found" / "not registered" / …)	critical
SEC-28	A burst of bad login attempts never authenticates and never 5xxs	6 wrong-credential logins → each not 200, each <500, none issues a session	critical
SEC-29	The admin create-user endpoint is blocked for anon	`POST /auth/v1/admin/users` with the anon key → **401/403** (no account provisioning)	critical
SEC-30	Auth errors return JSON, never an HTML page or a stack trace	Login-error response is not `text/html`, no HTML doc / source path / stack leaked	—

Run

```
npx playwright test --project=security
```

To include `SEC-06` and `SEC-08`, point `ORGANUZ_API_BASE_URL` at a disposable backend and acknowledge that exact origin:

```
SECURITY_WRITE_PROBES=true \
SECURITY_WRITE_TARGET=https://disposable-project.supabase.co \
ORGANUZ_API_BASE_URL=https://disposable-project.supabase.co \
npx playwright test --project=security
```

Notes

- **Default CI is non-destructive.** The only active write targets a non-existent row (`NON_EXISTENT_ID`). Potentially mutating denial probes require the explicit disposable-target opt-in above. Never enable them against a real dataset or add fuzzing volume or a service-role key here.
- A failure is a **real finding** — the backend's security posture regressed. Investigate before touching the test; do not relax an assertion to go green.
- Endpoints, headers, the sensitive-key list, and the JWT decoder live in `tests/security/support/target.ts`.